



SnuQS: Scaling Quantum Circuit Simulation using Storage Devices

Daeyoung Park
Department of Computer Science and
Engineering
Seoul National University
Seoul, 08826, South Korea
daeyoung@aces.snu.ac.kr

Heehoon Kim
Department of Computer Science and
Engineering
Seoul National University
Seoul, 08826, South Korea
heehoon@aces.snu.ac.kr

Jinpyo Kim
Department of Computer Science and
Engineering
Seoul National University
Seoul, 08826, South Korea
jinpyo@aces.snu.ac.kr

Taehyun Kim
Department of Computer Science and
Engineering
Seoul National University
Seoul, 08826, South Korea
taehyun@snu.ac.kr

Jaejin Lee
Department of Computer Science and
Engineering
Department of Data Science
Seoul National University
Seoul, 08826, South Korea
jaejin@snu.ac.kr

Abstract

Since the state-of-the-art quantum computers are still noisy and error-prone, classical simulation of quantum circuits is essential in verifying/calibrating quantum computers and prototyping/debugging complex quantum algorithms. Classical simulation of large quantum systems is challenging due to its exponential increase in space and computation requirements. In this paper, we propose a full-state simulation framework, SnuQS. It exploits storage devices, such as HDDs and NVMe SSDs, to enlarge the available main memory capacity at a small cost. To achieve maximum I/O bandwidth, we propose an overlay-based memory management technique and optimization techniques. We also propose an I/O subsystem architecture that guarantees the maximum bandwidth of each storage device. We evaluate SnuQS on a 64-core CPU and 4-GPU system with 80 2TB HDDs and 10 4TB NVMe SSDs using quantum supremacy and quantum Fourier transform circuits. The experimental result indicates that SnuQS and the proposed I/O subsystem together is an effective and practical solution to scale the full-state simulation of large quantum circuits at about 300× lower cost than the DDR4 DRAM main-memory-only system.

CCS Concepts: • Computing methodologies → Massively parallel algorithms; Simulation tools.

Keywords: Quantum computing, Quantum circuit simulation, Memory overlays, I/O optimization

ACM Reference Format:

Daeyoung Park, Heehoon Kim, Jinpyo Kim, Taehyun Kim, and Jaejin Lee. 2022. SnuQS: Scaling Quantum Circuit Simulation using Storage Devices. In *2022 International Conference on Supercomputing (ICS '22)*, June 28–30, 2022, Virtual Event, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3524059.3532375>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICS '22, June 28–30, 2022, Virtual Event, USA

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9281-5/22/06...\$15.00

<https://doi.org/10.1145/3524059.3532375>

1 Introduction

Quantum computing is a new computing paradigm that exploits quantum-mechanical phenomena, such as *superposition* and *entanglement*. *Quantum circuits* are the most popular model that represents quantum algorithms. A quantum circuit consists of one or more *qubits* and an ordered sequence of *quantum gates* operating on them. Each quantum algorithm starts with an initial state that evolves into the final state by applying a sequence of quantum gates in the quantum circuit. The final state represents the probability *amplitudes* of exponentially many states. By *measurement*, the final state collapses into a single basis state following the probability amplitudes. This process repeats many times until the results are statistically significant.

Classical quantum circuit simulation (or just *quantum circuit simulation* in this paper) is to mimic quantum systems and their evolution using classical computers. In the era of Noisy Intermediate-Scale Quantum (NISQ[40]), physical quantum computers are vulnerable to noise. Quantum circuit simulation is the only way to develop, test, and verify quantum algorithms, and many quantum computing platforms emphasize high-performance quantum simulators[1, 9].

Quantum circuit simulation is extremely hard due to the exponential growth of memory space and computations[20, 21, 23, 32, 33, 37]. There are mainly two types of quantum circuit simulation methods: Schrödinger-style *full-state simulation* and *amplitude sampling*. Table 1 summarizes the differences between them. Recent hybrid approaches combine full-state simulation and amplitude sampling in which enough amplitudes, typically one million amplitudes, are computed faster than the pure amplitude sampling method[23, 32, 37].

In this paper, we focus on the full-state simulation for the following reasons: First, the full-state simulation produces the exact final

Table 1. Comparison between the full-state simulation and amplitude sampling methods.

	Full-state simulation	Amplitude sampling
Algorithm	Schrödinger-style	Tensor network contraction
Performance characteristics	Memory-intensive	Computation-intensive
Circuit preference	Small but deep circuits	Large but shallow circuits
Objective	Virtual quantum computer	Quantum chip validation

Table 2. Qubit simulation limits of the top four supercomputers in TOP500 as of November 2021.

Rank	Name	Processors	# of nodes	Total memory	Qubit limit
1	Fugaku	CPU	158,976	4,866 TB	48
2	Summit	CPU+GPU	4,608	2,802 TB	47
3	Sierra	CPU+GPU	4,320	1,382 TB	46
4	Taihulight	CPU	40,960	1,311 TB	46

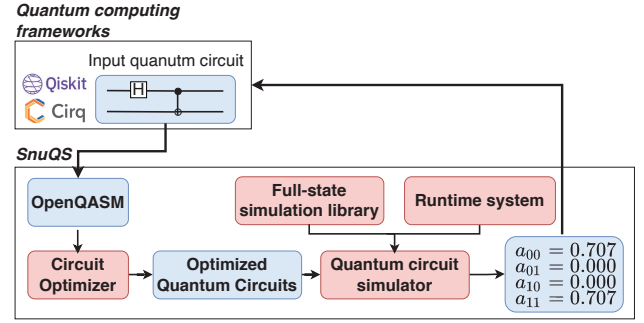
probability distribution. Hence it is easy to mimic a large real quantum computer using full-state simulation. Second, the full-state simulation produces intermediate results. Many quantum algorithms (e.g., error-correcting code and quantum teleportation[11, 12, 15, 43]) and debugging procedures (e.g., quantum assertion checking[27]) require intermediate results. Finally, full-state simulation is more efficient for deep quantum circuits than amplitude sampling. Many useful quantum circuits are deep or show better performance as they become deep (e.g., VQE[39]).

The main challenge in the full-state simulation is the exponential growth of memory space. An n -qubit quantum circuit requires $O(2^n)$ memory space to store all amplitudes in its state. Previous approaches make use of large-scale supercomputers to handle the exponential growth problem where the size of systems ranges from thousands to tens of thousands of nodes[25, 43]. Table 2 lists the top four supercomputers as of November 2021 in TOP500[5]. It shows their estimated qubit limits of full-state simulation in the double-precision floating-point format. Each has more than one petabyte of main memory, but the simulation limit is less than 50 qubits.

However, it is hard and expensive for an average user to access such supercomputers for quantum circuit simulation. Moreover, they consume a tremendous amount of energy. Instead, this paper proposes a full-state simulation framework that leverages storage devices, such as HDDs and NVMe SSDs, to scale quantum circuit simulation. We exploit storage devices, instead of DRAM, as secondary memory space to enlarge the main memory space and use just a single node to simulate large quantum circuits.

A typical way to extend the memory capacity by exploiting storage devices is to set up the virtual memory in the operating system. However, when used in the full-state simulation, the virtual memory suffers from performance degradation due to I/O bandwidth under-utilization and the overhead of frequent paging. Instead of using virtual memory, we propose an overlay-based memory management technique to hide the performance gap between the main memory and the storage devices. Overlays transfer a data block between the storage space and the main memory, replacing data in the memory space. The application explicitly controls the block transfer. Unlike other general-purpose applications, full-state simulation memory access patterns are simple and predictable. Thus memory overlays are very well suitable for full-state simulation. Indeed, an efficient memory and I/O management policy are required for overlays to circumvent performance degradation due to heavy I/O[17, 31].

Figure 1 shows the proposed framework *SnuQS* (SNU Quantum simulator using Storage devices). It consists of a circuit optimizer, a full-state simulation library, a runtime system, and a quantum circuit simulator. *SnuQS* takes a quantum circuit and the system description as input and produces all amplitudes stored in the storage space.

**Figure 1.** SnuQS framework.

The quantum circuit optimizer analyzes and transforms the input circuit into an equivalent optimized circuit with better simulation performance. Its optimizations include well-known optimization techniques, such as cache blocking[41] and gate fusion[9]. It also performs *SnuQS* sub-circuit partitioning, which is later used in the quantum circuit simulator. The full-state simulation library implements the operation of each gate. It parallelizes and vectorizes the gate operations using OpenMP[19] for CPUs and CUDA[35] for GPUs. The runtime system manages I/O and GPU commands based on CUDA[35] and Linux *io_uring* system calls[2]. Finally, the simulator implements overlays by calling the runtime system interface and simulates the quantum circuit by calling the gate functions. It overlaps the CPU/GPU computation, CPU/GPU communication, and block I/O.

Exploiting storage devices in quantum circuit simulation has not been explored yet except for analytical estimation in the hybrid method[37]. To the best of our knowledge, our proposal is the first work that uses storage devices to scale full-state simulation with reasonable performance. The contributions of this paper are summarized as follows:

- We propose a state amplitude layout representation based on qubit permutations. A qubit permutation determines an amplitude layout in the storage space and main memory. It also determines the block I/O pattern.
- We propose a sub-circuit partitioning technique that decomposes a quantum circuit into sub-circuits to maximize in-memory computation. Storage I/O occurs only at the beginning and at the end of an iteration of a sub-circuit.
- We propose a memory overlays technique for full-state simulation. The proposed three-step qubit permutation algorithm makes the simulation algorithm read and write the entire storage space only once at each sub-circuit boundary with no I/O performance degradation.
- We propose a quantum gate scheduling technique that balances computation between sub-circuits to maximize overlapping computation and communication.
- We propose an I/O subsystem architecture with many storage devices to implement efficient overlays and improve the I/O bandwidth.
- We compare *SnuQS* with state-of-the-art classical quantum circuit simulators, such as QuEST[28], IBM Qiskit Aer[9], and Google Cirq qsim[1].
- We implement *SnuQS* and the I/O subsystem. We evaluate *SnuQS* with the quantum supremacy circuit (QSC)[13, 14] and quantum Fourier transform circuit (QFT)[18, 29].

The experimental result shows that the in-memory simulation performance of SnuQS is comparable to the state-of-the-art simulators, such as QuEST, Qiskit Aer, and Cirq qsim. Using a single node machine with 4 NVIDIA GTX 3090 GPUs and 80 2TB HDDs, simulations of a 42-qubit QSC of depth 1+48+1 and a QFT of depth 84 take 16.72 and 7.44 hours, respectively. Using 10 4TB NVMe SSDs, simulations of a 39-qubit QSC of depth 1+48+1 and a QFT of depth 78 take 41.4 and 9.6 minutes, respectively. The simulation scale of SnuQS is infeasible for the current state-of-the-art quantum simulator on machines of the same scale. We estimate it takes a few years with virtual memory. The result indicates that SnuQS is an efficient way to scale the classical-quantum circuit simulation.

2 Quantum Circuit Model

In this section, we describe the quantum circuit and quantum gate operations.

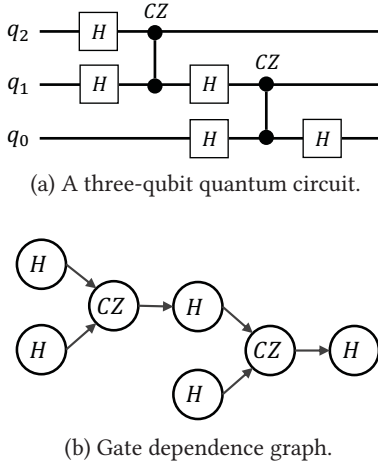


Figure 2. A 3-qubit quantum circuit and its dependence graph.

A quantum circuit consists of a set of qubits and an ordered sequence of quantum gates. Figure 2(a) shows a three-qubit quantum circuit with seven gates. We assume that we have a universal quantum gate set consisting of single-qubit gates and two-qubit gates without loss of generality[10].

Single-qubit gate operations. Let G be a single-qubit gate (i.e., a 2×2 unitary matrix). Then the operator $U_G^j = U_0 \otimes U_1 \otimes \dots \otimes U_{n-1}$ performs the single-qubit operation G on qubit q_j in an n -qubit system, where $U_j = G$ and $U_k = I$ such that $k \neq j$. Suppose that the state vector $|\psi\rangle$ is given by

$$|\psi\rangle = C_{00\dots 0} |00\dots 0\rangle + C_{00\dots 1} |00\dots 1\rangle + \dots + C_{11\dots 1} |11\dots 1\rangle.$$

Then, in general, U_G^j transforms the state $|\psi\rangle$ as follows[21]:

$$\begin{aligned} |\psi'\rangle &= U_G^j |\psi\rangle \\ &= C'_{00\dots 0} |00\dots 0\rangle + C'_{00\dots 1} |00\dots 1\rangle + \dots + C'_{11\dots 1} |11\dots 1\rangle, \end{aligned}$$

where

$$\begin{bmatrix} C'_{b_{n-1}b_{n-2}\dots 0_j\dots b_0} \\ C'_{b_{n-1}b_{n-2}\dots 1_j\dots b_0} \end{bmatrix} = G \begin{bmatrix} C_{b_{n-1}b_{n-2}\dots 0_j\dots b_0} \\ C_{b_{n-1}b_{n-2}\dots 1_j\dots b_0} \end{bmatrix} \quad (1)$$

such that $b_k \in \{0, 1\}$ and $k \in \{0, 1, \dots, n-1\}$. 0_j and 1_j stand for 0 and 1 at the j th position, respectively.

Two-qubit gate operations. Consider a two-qubit gate (i.e., a 4×4 unitary matrix) Q . Let $U_Q^{c,j}$ with $j < c$ be the $n \times n$ unitary operator that performs the two-qubit gate operation Q on the j th and c th qubits in an n -qubit system. Then, in general, $U_Q^{c,j}$ transforms the state $|\psi\rangle$ as follows:

$$\begin{aligned} |\psi'\rangle &= U_Q^{c,j} |\psi\rangle \\ &= C'_{00\dots 0} |00\dots 0\rangle + C'_{00\dots 1} |00\dots 1\rangle + \dots + C'_{11\dots 1} |11\dots 1\rangle, \end{aligned}$$

where

$$\begin{bmatrix} C'_{b_{n-1}\dots 0_c\dots 0_j\dots b_0} \\ C'_{b_{n-1}\dots 0_c\dots 1_j\dots b_0} \\ C'_{b_{n-1}\dots 1_c\dots 0_j\dots b_0} \\ C'_{b_{n-1}\dots 1_c\dots 1_j\dots b_0} \end{bmatrix} = Q \begin{bmatrix} C_{b_{n-1}\dots 0_c\dots 0_j\dots b_0} \\ C_{b_{n-1}\dots 0_c\dots 1_j\dots b_0} \\ C_{b_{n-1}\dots 1_c\dots 0_j\dots b_0} \\ C_{b_{n-1}\dots 1_c\dots 1_j\dots b_0} \end{bmatrix} \quad (2)$$

Two-qubit controlled gate operations. Suppose that Q in Equation 2 is a controlled- D gate such that D is an arbitrary 2×2 unitary operator. Also suppose that it performs the controlled- D operation on a target qubit q_j and a control qubit q_c s.t. $j < c$. It operates only when q_c is set to $|1\rangle$. It does nothing when q_c is set to $|0\rangle$. Thus, Equation 2 becomes

$$\begin{aligned} \begin{bmatrix} C'_{b_{n-1}\dots 0_c\dots 0_j\dots b_0} \\ C'_{b_{n-1}\dots 0_c\dots 1_j\dots b_0} \end{bmatrix} &= \begin{bmatrix} C_{b_{n-1}\dots 0_c\dots 0_j\dots b_0} \\ C_{b_{n-1}\dots 0_c\dots 1_j\dots b_0} \end{bmatrix} \\ \begin{bmatrix} C'_{b_{n-1}\dots 1_c\dots 0_j\dots b_0} \\ C'_{b_{n-1}\dots 1_c\dots 1_j\dots b_0} \end{bmatrix} &= D \begin{bmatrix} C_{b_{n-1}\dots 1_c\dots 0_j\dots b_0} \\ C_{b_{n-1}\dots 1_c\dots 1_j\dots b_0} \end{bmatrix} \end{aligned} \quad (3)$$

Diagonal gates. When the gate is diagonal (i.e., its matrix representation is a diagonal matrix), the matrix-vector multiplication in Equation 1 and Equation 2 becomes a vector-vector Hadamard product (i.e., element-wise vector multiplication denoted $\odot$):

$$\begin{bmatrix} C'_{b_{n-1}b_{n-2}\dots 0_j\dots b_0} \\ C'_{b_{n-1}b_{n-2}\dots 1_j\dots b_0} \end{bmatrix} = \begin{bmatrix} G_{11} \\ G_{22} \end{bmatrix} \odot \begin{bmatrix} C_{b_{n-1}b_{n-2}\dots 0_j\dots b_0} \\ C_{b_{n-1}b_{n-2}\dots 1_j\dots b_0} \end{bmatrix} \quad (4)$$

and

$$\begin{bmatrix} C'_{b_{n-1}\dots 0_c\dots 0_j\dots b_0} \\ C'_{b_{n-1}\dots 0_c\dots 1_j\dots b_0} \\ C'_{b_{n-1}\dots 1_c\dots 0_j\dots b_0} \\ C'_{b_{n-1}\dots 1_c\dots 1_j\dots b_0} \end{bmatrix} = \begin{bmatrix} Q_{11} \\ Q_{22} \\ Q_{33} \\ Q_{44} \end{bmatrix} \odot \begin{bmatrix} C_{b_{n-1}\dots 0_c\dots 0_j\dots b_0} \\ C_{b_{n-1}\dots 0_c\dots 1_j\dots b_0} \\ C_{b_{n-1}\dots 1_c\dots 0_j\dots b_0} \\ C_{b_{n-1}\dots 1_c\dots 1_j\dots b_0} \end{bmatrix} \quad (5)$$

Circuit depth. Suppose that each gate in a quantum circuit represents a node in a graph, and each line between gates represents a left-to-right directed edge. Then, the resulting graph represents a gate dependence graph of the quantum circuit. Figure 2(b) shows that each CZ gate depends on the two H gates in Figure 2(a). We define the *depth* of a quantum circuit to be the number of nodes in the longest path in the dependence graph. The depth of the circuit in Figure 2(a) is five. Any gate sequence in the topological sort order of the dependence graph produces the same correct result.

3 Simulation using Storage Devices

In this section, we propose the techniques to scale the full-state simulation using storage devices, such as HDDs and NVMe SSDs.

3.1 Notations and Parameters

We assume that state amplitudes are buffered in the main memory without loss of generality. The state buffer is named s and is addressable in the unit of amplitude, and the storage space is block-addressable. Note that a storage device, such as HDDs and NVMe

SSDs, is a block device that reads or writes one block of data (e.g., 512 bytes) at a time. We assume that the size of s is 2^m amplitudes and the block size is 2^b amplitudes s.t. $m > b$. The main memory size S_M satisfies $2^m < S_M < 2^m + 1$ because of the data other than amplitudes.

3.2 Schrödinger-style Algorithms

Given an n -qubit quantum circuit, the Schrödinger-style simulation keeps 2^n complex amplitudes in the main memory buffer s and updates them in place in a gate-by-gate manner[20, 21]. Suppose that a given n -qubit quantum circuit consists of universal single-qubit or two-qubit gates[10]. Also, the amplitudes of states are stored in the main memory in the order of $|00 \dots 00\rangle, |00 \dots 01\rangle, \dots, |11 \dots 11\rangle$.

Simulation of gates. The simulation of a single-qubit gate g is based on Equation 1. Basically, the Schrödinger-style algorithm reads and writes all amplitudes in s . If $g.targets[0] = q_j$, it reads one by one a pair of amplitudes that are 2^j -amplitude apart. It multiplies the 2×2 gate matrix $g.M$ with the pair and writes back the two resulting amplitudes to s . For example, consider a five-qubit circuit and $g.targets[0] = q_4$. The algorithm iterates over the pairs of amplitudes in the order of the corresponding pairs of basis vectors: $(|040000\rangle, |140000\rangle), (|040001\rangle, |140001\rangle), (|040010\rangle, |140010\rangle), \dots, (|041111\rangle, |141111\rangle)$.

The same argument applies to the simulation of a two-qubit gate g , but the simulation algorithm reads four amplitudes at a time based on Equation 2. When $g.targets[0] = q_j$ and $g.targets[1] = q_c$ are the target qubits of the gate, it reads two pairs of amplitudes (C_{00}, C_{01}) and (C_{10}, C_{11}) at a time, where C_{00} and C_{10} are 2^c -amplitude apart and C_{00} and C_{01} (also C_{10} and C_{11}) are 2^j -amplitude apart. Then, it multiplies them with a 4×4 matrix $g.M$ and writes back the resulting amplitudes to s . For example, consider a five-qubit quantum system and $g.targets[1] = q_4$ and $g.targets[0] = q_2$. The algorithm iterates over the quadruples of amplitudes in the order of the corresponding quadruples of basis vectors: $(|040020\rangle, |040120\rangle, |140020\rangle, |140120\rangle), (|040021\rangle, |040121\rangle, |140021\rangle, |140121\rangle), \dots, (|041021\rangle, |041121\rangle, |141021\rangle, |141121\rangle)$.

Controlled gates and diagonal gates. Two-qubit controlled gates, such as $CNOT$ and CZ , can be optimized based on Equation 3. A two-qubit controlled gate accesses only half of the amplitudes. When the gate is diagonal, the simulation algorithm can be further optimized based on Equation 4 and Equation 5 using element-wise vector-vector multiplication.

3.3 Memory Overlays using Storage Devices

We define an *amplitude layout* for an n -qubit quantum circuit as an order of all of the 2^n amplitudes in a state. The goal of SnuQS memory overlays is that once the main memory state buffer s is filled with consecutive amplitude blocks according to the amplitude layout, SnuQS applies gates to all the amplitudes in s , as many as possible, without incurring extra block I/O to load missing amplitudes in s . For example, to apply the gate matrix without incurring any extra block I/O, the two amplitudes $C_{b_{n-1}b_{n-2} \dots 0j \dots b_0}$ and $C_{b_{n-1}b_{n-2} \dots 1j \dots b_0}$ in Equation 1 should reside in s at the same time. To avoid and minimize block I/O occurring in the full-state simulation using overlays, we propose an amplitude layout transformation framework using *qubit permutations*.

3.4 Qubit Permutations

Qubit positions. For clarity, we define a qubit *index* and a qubit *position*. A qubit index is a unique identifier (ID) of each qubit. The qubit indices of an n -qubit circuit are given by $n-1, n-2, \dots, 1$ and 0. We denote a qubit with q subscripted with its index (e.g., q_3). There are $n!$ ways of ordering the n qubits. Each different qubit order is a permutation of the n qubits $((q_4q_3q_2q_1q_0), (q_3q_2q_1q_0q_4), \text{etc.})$

A qubit *position* indicates where each qubit is located in a given permutation or a given quantum circuit (Figure 3). We define a function $pos_\pi(q)$ to obtain the position of qubit q in a given qubit permutation π . For example, when $\pi = (q_0q_3q_2q_1q_4)$, $pos_\pi(q_0) = 4$, $pos_\pi(q_1) = 1$, $pos_\pi(q_2) = 2$, $pos_\pi(q_3) = 3$, and $pos_\pi(q_4) = 0$.

Representing amplitude layouts. In our method, a qubit permutation determines an amplitude layout. We define a function ξ whose value is the position (from 0 to $2^n - 1$) of amplitude in the amplitude layout. When the basis ket for the amplitude, $|b_{n-1}b_{n-2} \dots b_0\rangle$ and a qubit permutation π are given:

$$\xi(b_{n-1}b_{n-2} \dots b_0, \pi) = \sum_{j=0}^{n-1} b_j \cdot 2^{pos_\pi(q_j)}. \quad (6)$$

For example, when $\pi = (q_4q_3q_2q_1q_0)$, the order of amplitudes in the layout is the lexicographic order of their corresponding basis kets. Figure 6(b) shows such a layout. The amplitude layouts (c), (d), and (e) in Figure 6 also show different layouts for different qubit permutations.

Local and non-local gates. Suppose that we have an n -qubit quantum circuit s.t. $m < n$. We categorize qubits in two classes depending on their *positions* in a permutation. If a qubit's position is less than m , the qubit is *local* to the permutation (i.e., w.r.t. the corresponding layout). Otherwise, it is *non-local*. For example, consider a quantum circuit in Figure 3(a). The permutation, in this case, is $(q_4q_3q_2q_1q_0)$. When $m = 4$, q_0, q_1, q_2 , and q_3 are local and q_4 is non-local.

When the target qubits of a gate are all local, the gate is called a *local gate*. Otherwise, the gate is a *non-local gate*. Once s is filled with consecutive amplitude blocks according to a given amplitude layout, the problem of the non-local gate is that we may not apply the gate to its target qubits without incurring any extra block I/O. For example, when $j \geq m$, the two amplitudes $C_{b_{n-1}b_{n-2} \dots 0j \dots b_0}$ and $C_{b_{n-1}b_{n-2} \dots 1j \dots b_0}$ in Equation 1 cannot reside in s at the same time because they are 2^m amplitudes apart in the given amplitude layout. Note that the size of s is 2^m amplitudes. Thus, we need an extra block I/O operation to load the missing amplitude in s .

A diagonal gate g is always local even if it has non-local target qubits. The amplitudes are processed based on Equation 4 or Equation 5. Once s is filled with amplitude blocks, there reside in s all necessary amplitudes to apply $g.M$. For a single-qubit diagonal gate, Equation 4 is written for a single amplitude in the following way:

$$C'_{b_{n-1}b_{n-2} \dots b_j \dots b_0} = \begin{cases} g.M_{11} \cdot C_{b_{n-1}b_{n-2} \dots b_j \dots b_0} & \text{if } b_j = 0 \\ g.M_{22} \cdot C_{b_{n-1}b_{n-2} \dots b_j \dots b_0} & \text{if } b_j = 1 \end{cases} \quad (7)$$

The Equation 5 can be written for a single amplitude in a similar way. Thus, a diagonal gate processes the amplitudes in s one by one without any extra block I/O.

In the quantum circuit of Figure 3(a), $G_1, G_2, G_3, G_4, CZ_1, CZ_2, G_6, G_7$, and G_8 are local gates, but $CNOT_1$ and G_5 are non-local gates (assuming G_5 is non-diagonal).

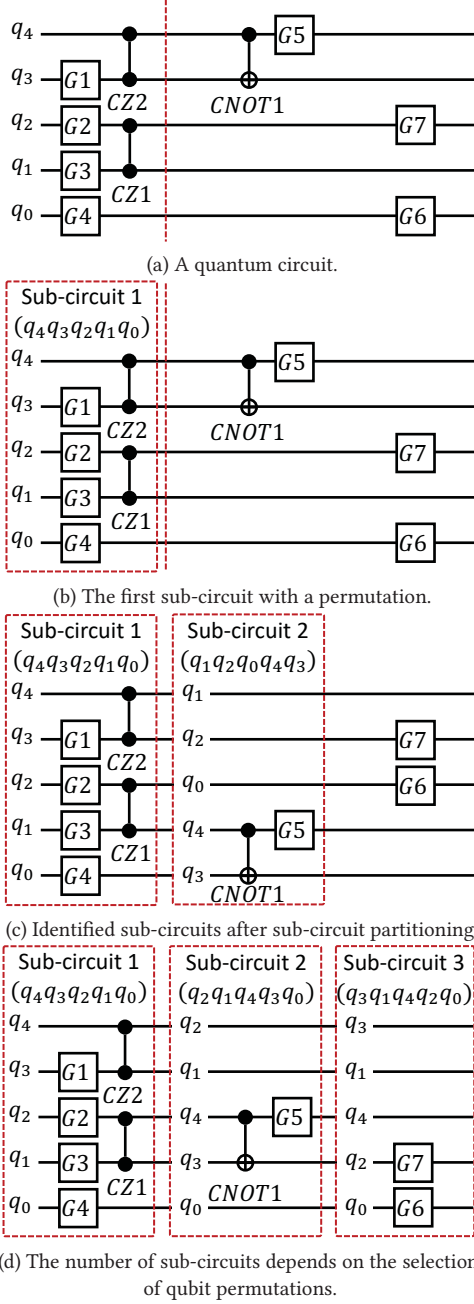


Figure 3. Sub-circuit partitioning with permutations.

Automatic sub-circuit partitioning. In Figure 3(a), the red dotted line divides local gates on the left and non-local gates on the right w.r.t. $(q_4q_3q_2q_1q_0)$. To handle the non-local gates, we need to change the amplitude layout at the dotted line. The changed layout makes the non-local gates (e.g., $CNOT1$ and $G5$) local w.r.t. the changed layout.

In general, we vertically partition the input circuit into a set of sub-circuits and associate each sub-circuit with a qubit permutation that determines the amplitude layout for the sub-circuit. Two adjacent sub-circuits have different permutations. The gates in a sub-circuit are all local w.r.t. its associated qubit permutation. There

is no need to change the amplitude layout during the simulation of a sub-circuit (i.e., no block I/O).

The sub-circuit partitioning algorithm continues adding in *subcircuit* local gates w.r.t. *perm* until it meets a non-local gate w.r.t. *perm*. The non-local gate starts a new sub-circuit and produces a new qubit permutation associated with the new sub-circuit.

For example, Figure 3(b) shows the first sub-circuit obtained after applying the subcircuit partitioning algorithm. When the algorithm meets a non-local gate ($CNOT1$) w.r.t. $(q_4q_3q_2q_1q_0)$, it stops building the first sub-circuit and starts a new sub-circuit. Note that CZ gates are diagonal and local w.r.t. any permutation. In Figure 3(c), $CNOT1$ starts a new sub-circuit and produces a new qubit permutation $(q_1q_2q_0q_4q_3)$. Finally, the quantum circuit in Figure 3(c) is partitioned into two sub-circuits: sub-circuit 1 and sub-circuit 2.

The number of sub-circuits for a given quantum circuit depends on the permutations we select. Figure 3(d) shows sub-circuits with different permutations other than those in Figure 3(c). As a result, the total amount of block I/O significantly increases compared with the partition in Figure 3(c).

Permutation selection. Simulating a sub-circuit requires all amplitudes in the storage devices to be read and written once, and the total I/O amount is proportional to the number of sub-circuits. But the number of sub-circuits depends on the permutations we select, as shown in Figure 3(c) and Figure 3(d). The number of sub-circuits can be reduced dramatically by selecting adequate permutations. Since the problem is complicated and there is no known optimal algorithm to find an optimal permutation among $n!$ different permutations, we rely on heuristics. The permutation selection algorithm determines the next permutation by looking ahead at the patterns of the remaining gates.

Suppose that g is the gate that starts a new subcircuit. That is it is a non-local gate w.r.t. the associated permutation *perm* to the previous subcircuit. Also, suppose that *gate_list* contains the remaining gates to be processed. The heuristic permutation selection algorithm works as follows:

- For each qubit q , it counts in *gate_list* the number of target local gates w.r.t. *perm* and the number of target non-local gates w.r.t. *perm*.
- If a qubit q is g 's target qubit, set its local gate count and non-local gate count to ∞ .
- It returns the new permutation *new_perm* with the qubit ordering from the right in the permutation in a non-increasing order of the non-local gate counts and then a non-increasing order of the local gate counts.

We expect that the more non-local gates target a qubit, the better localizing the qubit in the next sub-circuit.

3.5 SnuQS Simulation Algorithm

Suppose that SnuQS simulates an n -qubit quantum circuit and 2^m is the size of main memory buffer s . Also suppose that *subcircuit_list* contains the subcircuits after the subcircuit partitioning. The simulation algorithm iterates over the sequence of sub-circuits in *subcircuit_list*. For each sub-circuit, it processes 2^m amplitudes (the size of s) at a time until all 2^n amplitudes are processed. It first reads 2^m amplitudes from the storage space to s . The position of each amplitude in s depends on the associated permutation to the sub-circuit. Then, it simulates the gates in the sub-circuit using gate simulation functions. These functions are

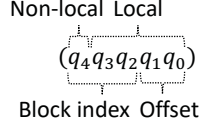


Figure 4. Position fields.

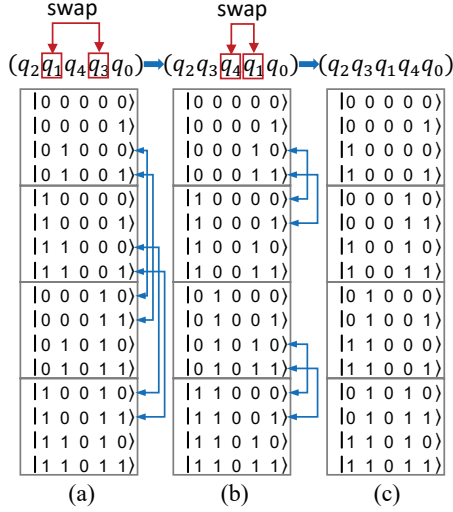


Figure 5. Implementation of layout changes caused by the in-memory transformations.

based on Equation 1 and Equation 2 and can be further optimized for diagonal gates based on Equation 4 and Equation 5. Then, it writes the 2^m amplitudes in s to the storage space. The final state amplitudes are stored in the storage space.

Three-step layout transformation. To minimize storage I/O, we try to make each sub-circuit simulation reads and writes all amplitudes in the storage space only once. When the storage access unit is an amplitude to implement the associated permutation to each sub-circuit, it incurs huge overhead because a storage device is a block device. The block size is much bigger than the amplitude size. To exploit the benefits of a block device, we propose a three-step layout transformation technique to implement the permutation.

Suppose that we are given a five-qubit circuit, the size of s is $2^4 = 16(m = 4)$ amplitudes, and the I/O block size is $2^2 = 4$. Then, as shown in Figure 4(a), there are four different fields in a qubit permutation: non-local qubits, local qubits, block index, and offset in a block. A layout transformation transforms a permutation $perm_s$ to another permutation $perm_d$ with the same length, denoted by $perm_s \rightarrow perm_d$. When it permutes only local qubits in $perm_s$ and obtains $perm_d$, we call it an *in-memory transformation* because changing amplitude layouts according to the transformation can be done in s without extra block I/O. On the other hand, when a transformation permutes qubits only in the block index field of $perm_s$ and obtains $perm_d$, we call it a *block-index transformation*. The three-step transformation process transforms a given permutation $perm_s$ into a final permutation $perm_d$ through three intermediate transformation steps: an in-memory transformation (the first in-memory transformation), a block-index transformation,

and then an in-memory transformation (the second in-memory transformation) again.

For example, suppose that the size of s is $2^4 = 16(m = 4)$. From a given permutation $(q_2 q_1 q_4 q_3 q_0)$, we obtain $(q_3 q_1 q_4 q_2 q_0)$ using the three-step transformation process: $(q_2 q_1 q_4 q_3 q_0) \xrightarrow{\text{first in memory}} (q_2 q_3 q_1 q_4 q_0) \xrightarrow{\text{block index}} (q_3 q_1 q_2 q_4 q_0) \xrightarrow{\text{second in memory}} (q_3 q_1 q_4 q_2 q_0)$.

We can transform any permutation into another permutation using the three-step transformation process with one restriction on the block size parameter b . Suppose we are given n -qubit quantum circuit, and 2^m is the size of s . There are $n - m$ non-local qubit positions and we have to reserve the leftmost $n - m$ local qubit positions for the block-index transformation. Thus, b has to satisfy $b < m - (n - m) = 2m - n$.

We can decomposes any in-memory transformation $perm_s \rightarrow perm_d$ into a sequence of in-memory transformations where each of them swaps only two qubits. For example, an in-memory transformation $(q_2 q_1 q_4 q_3 q_0) \rightarrow (q_2 q_3 q_1 q_4 q_0)$ is decomposed to a sequence of three in-memory transformations $(q_2 q_1 q_4 q_3 q_0) \rightarrow (q_2 q_3 q_4 q_1 q_0) \rightarrow (q_2 q_3 q_1 q_4 q_0) \rightarrow (q_2 q_3 q_1 q_4 q_0)$ (one of them is an identity transformation). The layout changes and the processes of swapping amplitudes according to the in-memory transformations are shown in Figure 5. Since $(q_2 q_3 q_1 q_4 q_0) \rightarrow (q_2 q_3 q_1 q_4 q_0)$ is an identity transform, it does not change the layout.

Amplitude layout changes. Consider the amplitude layout changes in Figure 6 when simulating the three sub-circuits in Figure 3(d). Suppose that 2^5 and 2^4 are the sizes of storage space and the main-memory state buffer s , respectively. Also suppose that the initial amplitude layout in the storage space is $(q_4 q_3 q_2 q_1 q_0)$. The SnuQS simulation algorithm processes a total of 2^5 amplitudes to simulate a sub-circuit, 2^4 amplitudes at a time. The algorithm reads the 2^4 amplitudes from the storage space by invoking `readBlocks()` that reads storage blocks to s using the block-index permutation. In this case, the block-index transformation is $(q_4 q_3 q_2 q_1 q_0)$ and resulting amplitude layout in s is shown in Figure 6(a).

Suppose that the size of memory buffer s is equal to the size of storage space. Let $y_{n-1} y_{n-2} \dots y_b$ be the binary representation of a block index j in s and $\pi = perm_{bi}$ is the block-index permutation. Then the corresponding storage block index to read for block j in s is determined by

$$\xi(y_{n-1} y_{n-2} \dots y_b, \pi) = \sum_{j=b}^{n-1} y_j \cdot 2^{pos_{\pi(q_j)} - b}. \quad (8)$$

Note that the block-index transformation is implemented by changing the order of storage blocks to be read into s . Then, `readBlocks()` applies the second in-memory transformation $(q_4 q_3 q_2 q_1 q_0) \rightarrow (q_4 q_3 q_2 q_1 q_0)$ to s and obtains the layout $(q_4 q_3 q_2 q_1 q_0)$ shown in Figure 6(b). This is the layout for simulating sub-circuit 1.

After the simulation of sub-circuit 1 on s , The simulation algorithm invokes `writeBlocks()`. It performs the first in-memory transformation $(q_4 q_3 q_2 q_1 q_0) \rightarrow (q_4 q_2 q_3 q_1 q_0)$ to change the layout shown in Figure 6(b) to the layout $(q_2 q_1 q_4 q_3 q_0)$ in Figure 6(c) for the simulation of sub-circuit 2. Then, it stores the memory blocks in s to the storage space. The simulation continues until all the amplitudes in the storage space are processed for the sub-circuit 1 simulation. Then, the simulation algorithm starts simulating sub-circuit 2.

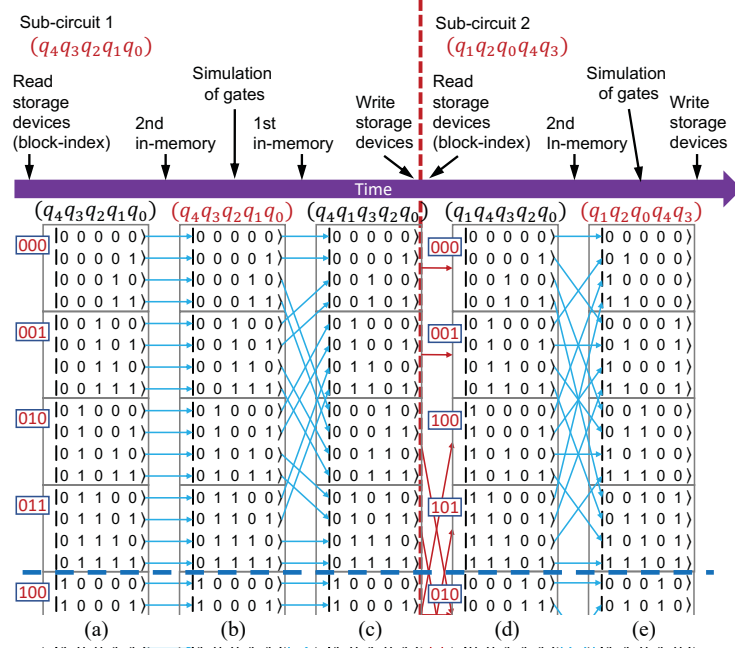


Figure 6. Amplitude layout changes when simulating the sub-circuits in Figure 3(c).

As shown in Figure 6(d) and (e), the simulation of sub-circuit 2 is similar to that of sub-circuit 1. The calls to `writeBlocks()` for sub-circuit 2 do not perform the first in-memory transformation because there is no sub-circuit after sub-circuit 2. The amplitude layout in the storage space is $(q_1q_2q_0q_4q_3)$ after the simulation of sub-circuit 2.

Table 3. System Configurations

CPU	AMD EPYC 7702P (2.0GHz, 64 cores, 8 memory channels, PCIe 4.0 $\times 128$, AVX2 vector instruction set)
Main memory	8 $\times$ 64GB DIMM (DDR4, 3200MHz)
GPU	4 $\times$ NVIDIA GeForce RTX 3090 (GDDR6X 24GB, PCIe 4.0 $\times 16$)
RAID cards	2 $\times$ Broadcom MEGARAID 9580 8i8e (PCIe 4.0 $\times 8$, each configured to RAID0 with 40 HDDs)
HDDs	80 $\times$ Seagate BarraCuda (2TB, 7200RPM, SATA3 (6Gb/s))
NVMes	10 $\times$ Sabrent Rocket 4 PLUS(4TB, $\times 4$ PCIe Gen 4 (8GB/s))

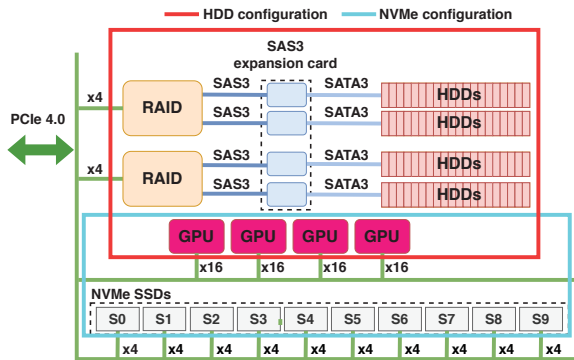


Figure 7. GPUs, HDDs and NVMeS connected to the PCIe 4.0 bus.

4 Implementations

In this section, we describe the proposed system architectures, runtime system and implementations of SnuQS.

4.1 The System Architecture

We simulate quantum circuits using two systems: a CPU system and a GPU system. The specific configurations of our target systems are shown in Table 3. The system consists of a multicore CPU, four GPUs, and many storage devices. For the HDD configuration, we use 80 HDDs with two RAID cards. Each RAID card provides two external SAS3 $\times 4$ ports, and we connect two SAS3 expansion cards through the ports. Each SAS3 expansion card is linked with 20 HDDs. Thus, each RAID card is connected to 40 HDDs, thus 80 HDDs in total. For the NVMe configuration, we use 10 NVMeS directly attached to M.2 and PCIe bus.

The system accommodates the maximum bandwidth of each storage device attached to the system. Each HDD provides a maximum bandwidth of 220MB/s for both sequential reads and writes. SATA3 bandwidth, 768MB/s, can provide enough bandwidth for an HDD. The SAS3 $\times 4$ interface provides a maximum bandwidth of 6GB/s. Thus, accommodating 20 HDDs to a single SAS3 $\times 4$ line (i.e., 220MB/s $\times 20 = 4,400\text{MB/s} < 6\text{GB/s}$) does not cause any bandwidth issue. Since the bandwidth of PCIe 4.0 $\times 8$ (16GB/s) exceeds the maximum bandwidth of 40 HDDs (8.8GB/s), the proposed I/O subsystem accommodates the maximum possible bandwidth for all HDDs. For NVMe devices, the system easily provides maximum bandwidth because they are directly attached to the PCIe bus without sharing any PCIe lanes.

4.2 SnuQS Runtime System

Figure 8 shows the SnuQS runtime system. The runtime system consists of I/O and GPU subsystems. Both I/O and GPU subsystems have request-completion queue pairs and the corresponding dispatcher threads that manage the queue pairs and execute requests. Each dispatcher thread keeps polling the request queue. When a

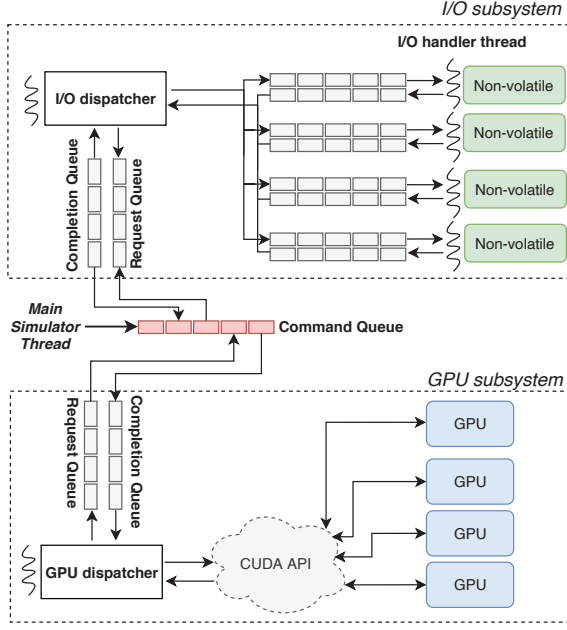


Figure 8. Design of SnuQS runtime system.

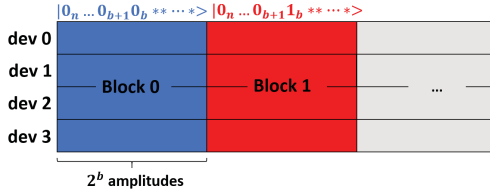


Figure 9. Parallel storage layout.

command is enqueued, it executes the command and enqueues it to the completion queue after completion. To maximize parallelism, all operations of the runtime system are asynchronous, and the main thread manages explicit synchronization.

A simulation process starts with the main simulator thread. It takes an optimized quantum circuit and simulates the circuit by continuously enqueueing either a gate operation command, a CPU-GPU communication command to the GPU subsystem, or an I/O operation command to the I/O subsystem. The I/O subsystem manages I/O operations using multiple *I/O handler* threads. An *I/O handler* thread resides in a CPU core with no migration. We use up to 64 I/O handlers. The number is equal to the maximum number of CPU cores. The storage devices are assigned to the *I/O handler* threads in a round-robin manner. The handler executes I/O operations using Linux `io_uring` system calls[2]. The *GPU subsystem* works similar to the I/O subsystem, but the dispatcher thread executes the command using the CUDA runtime.

4.3 Optimizations for Faster Simulation

I/O Parallelism. To improve I/O performance, we organize storage devices in a similar way to the RAID-0 configuration[36] but with a large block size (stripe) (Figure 9). When there are d storage devices with block size parameter b , we distribute each block of 2^b amplitudes across the d devices. Each device contains $\lceil 2^b/d \rceil$ amplitudes per block. Then SnuQS parallelizes I/O operations using this layout and the asynchronous I/O library.

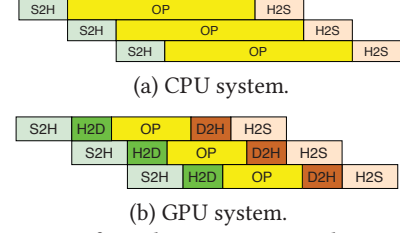


Figure 10. Data transfer and computation overlapping (triple buffering).

The size of any I/O operation is greater than or equal to 2^{b+4} bytes. The parameter b is configurable, but there is a restriction for performance. Since each I/O operation occurs in page-size granularity, each device had better contain more than 2^8 amplitudes per block, and hence $b \geq d \cdot 2^8$.

Data parallelism. Schrödinger-style algorithms are embarrassingly parallel[20, 21]. We parallelize the gate operations using OpenMP[19] and CUDA[35] for the CPU and the GPUs, respectively. For CPUs, we parallelize all loops in the gate functions using OpenMP. The number of workers is equal to the number of cores in the CPU. For GPUs, we build a CUDA kernel for each gate operation. We launch 2^{n-1} CUDA threads for a one-qubit gate, and each CUDA thread updates a pair of two amplitudes based on Equation 1. For a two-qubit gate, we launch 2^{n-2} CUDA threads.

Computation and I/O overlapping. To hide the heavy storage I/O overhead, SnuQS applies triple buffering using three main memory state buffers. While it performs gate operations on a buffer, it prefetches another buffer from the storage space and flushes the remaining buffer to the storage space. Figure 10(a) shows the pipeline of the computation and communication overlapping for the CPU simulation. *OP*, *S2H* and *H2S* stands for gate operations, I/O from the storage space to the host memory (read), and I/O from the host memory to the storage space (write), respectively. We apply a similar method to the GPU system but also overlap the data transfer between the CPU and the GPUs with gate operations. Figure 10(b) shows the pipeline, where *H2D* and *D2H* stand for the data transfer from the CPU to the GPU and the GPU to the CPU, respectively. Each GPU has a triple buffering pipeline in SnuQS implementation.

Load balancing between sub-circuits. To hide the I/O and communication (between CPU cores or GPUs), each sub-circuit needs to have enough computation. To balance the computation workload between sub-circuits, we compute the average operational intensity of all sub-circuits and make each have operational intensity similar to the average by moving gates across sub-circuits. The operational intensity is the ratio of floating-point operations to memory traffic [42].

5 Evaluation

In this section, we evaluate SnuQS using the HDD and NVMe configurations as in the previous section (Table 3).

5.1 Benchmark Applications

Table 4 summarizes target benchmark quantum circuits for evaluation: quantum Fourier transform circuits (QFTs)[18] and quantum supremacy circuits (QSCs)[13]. QFT is the most widely used quantum algorithm that has a quadratically increasing number of gates and linearly increasing depth to the number of qubits. We use QFTs with no final swap gates. QSCs are standard benchmarks to measure the performance of quantum circuit simulation. They are highly

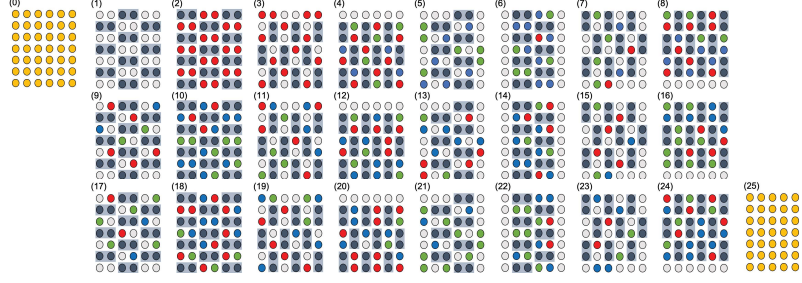


Figure 11. 7x6-qubit QS circuit with depth 1+24+1. Yellow: H gate, Indigo: CZ gate, Red: T gate, Blue: $\sqrt{X}$ gate, and Green: $\sqrt{Y}$ gate.

Table 4. Benchmark Applications and Simulation Time

Name	# of qubits	Depth	# of gates	Storage device	I/O space size	Simulation time
QSC-42-26	42	26	625	HDD	64TB	9.81 hours
QSC-42-50	42	50	1,172	HDD	64TB	16.72 hours
QFT-42-84	42	84	903	HDD	64TB	7.44 hours
QSC-39-26	39	26	582	NVMe	8TB	25.2 mins
QSC-39-50	39	50	1,063	NVMe	8TB	41.4 mins
QFT-39-78	39	84	780	NVMe	8TB	9.6 mins

entangled and have complex gate patterns. We generate QSCs by following the generating rules including the first and last Hadamard depth. Figure 11 shows a depth 26 (1+24+1) 42-qubit QSC used in the evaluation. We denote each circuit by its name, followed by the number of qubits and its depth. For example, QSC-42-50 stands for a 42-qubit QSC of depth 1+48+1. We simulate QSCs and QFTs up to 42 qubits using HDDs and 39 qubits using NVMe SSDs.

5.2 Simulation Time

Table 4 shows the simulation time of the benchmark circuits on the GPU system. The result shows that the SnuQS simulation time of QSC-42-50 using HDDs can be finished *within a day* (16.72 hours). The simulation of QFT-42-84 benchmark using HDDs is finished within 7.44 hours. It shows that using HDDs and a single node, we can simulate 42-qubit quantum circuits within a reasonable time. The result reveals that using NVMe SSDs shows significant speedup compared to using HDDs. Assuming that a 42-qubit quantum circuit takes 8 times longer simulation time than the corresponding 39-qubit circuit, the speedups of using NVMe SSDs over using HDDs are 2.91, 3.02, and 5.8, respectively.

We note that even if we can simulate up to 41-qubit quantum circuits using the NVMe SSDs in the evaluation system, the simulation performance significantly drops because of the SLC cache contention in the NVMe SSDs. NAND-flash technology increases the capacity by using multi-level cells (MLCs). To hide the slow bandwidth of MLCs, many SSDs adopt the hybrid architecture[45] that uses multi-level cells (SLCs) as a cache. The NVMe SSDs we use also have a hybrid architecture with about 512GB SLCs. We expect that SnuQS scales better with NVMe SSDs if we use high-end NVMe SSDs consisting only of SLCs.

5.3 Deep Circuits

Unlike the amplitude sampling method, the full-state simulation is robust to deep quantum circuits. The previous full-state simulation result of the deepest QSCs are those with 45 qubits/depth 26 and 49 qubits/depth 39 using 8,192 and 16,384 nodes, respectively[25, 32]. QFTs are deep circuits, and we use a depth 84

Table 5. Simulation Time on the CPU System

Name	# of qubits	Storage device	Compute device	Simulation time	GPU speedup over CPU
QSC-42-26	42	HDD	CPU	45.65 hours	4.65
QFT-42-84	42	HDD	CPU	42.87 hours	5.76

circuit. No amplitude sampling method simulates such deep and large QFTs as far as we know. Moreover, the simulation time of QSC-42-50 is about twice as much as QSC-42-26. It implies that SnuQS simulation time is linearly proportional to the circuit depth.

5.4 CPU versus GPU

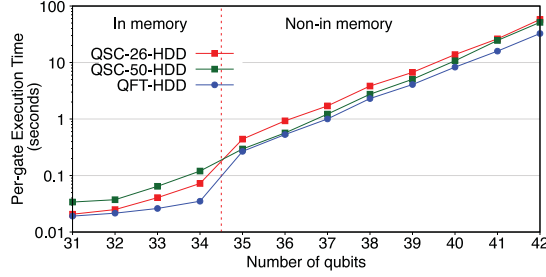
We evaluate QSC-42-26 and QFT-42-84 benchmarks using HDDs to compare simulations on the CPU and the GPU system. Table 5 shows the simulation times on the CPU system of the two benchmarks. SnuQS on the GPU system outperforms SnuQS on the CPU system in all cases achieving up to 5.76 $\times$ speedup.

We use an empirical Roofline model[42] and it shows that each gate is memory-bound both on the CPU and GPU systems. The significant performance difference between them comes from the memory bandwidth. Measured with Empirical Roofline Toolkit (ERT)[3], the CPU memory bandwidth is too small (142.8 GB/s). Thus the gate operation time dominates the simulation time. On the contrary, we use four GPUs, and their high memory bandwidth (4×819.2 GB/s) results in much faster gate operations. However, the storage I/O time still dominates the simulation time because the storage bandwidth (up to 1540 MB/s 61.2GB/s for HDDs and NVMe, respectively, measured with Fio[4]) is too small compared with the memory bandwidth (4×819.2 GB/s).

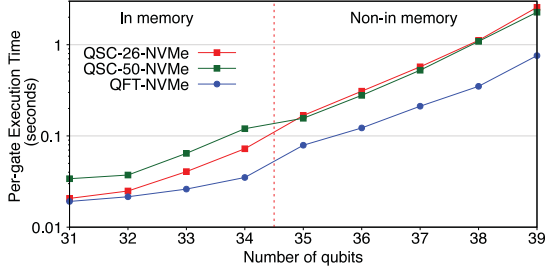
5.5 Scalability

Figure 12 shows the per-gate simulation time by varying the number of qubits in the benchmarks. To show the scalability of SnuQS, we use overlays even though the quantum circuits fit in the main memory for 31 - 34 qubits. Since each circuit has a different number of gates, we use the *per-gate* execution time (the total execution time divided by the number of gates). The y-axis is in the log scale, and the graph shows a nice straight curve in the sense that the simulation time is exponentially proportional to the number of qubits. But there is a slowdown at 34 qubits, especially for HDDs in Figure 12(a), but the slowdown is acceptable considering the bandwidth gap between CPU-GPU communication bandwidth (~ 64 GB/s) and I/O bandwidth (~ 16 GB/s).

We can simulate larger circuits using storage devices with larger capacity in a scalable way. Since the maximum capacity of current commercially available HDDs is 16TB (\$380[7]), we can attach a total of 80×16 TB= 1,280TB to the system. Assuming a simple



(a) Per-gate execution time using HDDs.



(b) Per-gate execution time using NVMe SSDs.

Figure 12. Per-gate execution time of SnusQS for a different number of qubits.**Table 6.** Memory Overlays versus Demand Paging (in hours)

Gate	QuEST (demand paging)	Qiskit (demand paging)	SnusQS (overlays)
H	23.89	23.13	1.44
CZ	9.74	8.90	0.67

exponential scale, we expect that QSC-45-26, QSC-45-50, and QFT-45-78 take 78.48 hours (3.27 days), 133.76 hours (5.57 days), and 59.52 hours (2.48 days), respectively. Using a single node, we can simulate 45-qubit quantum circuits *within a week*.

5.6 Cost-effectiveness

SnusQS is more cost-effective than the conventional methods using clusters. The costs of DDR4 DRAM (\$385 per 64GB DIMM[6]), NVMe(\$390 per 16TB HDD[7]) and HDD (\$390 per 16TB HDD[7]) per GB are \$6.01, \$0.19, and \$0.02, respectively. We conservatively estimate that we can simulate a 45-qubit QSC at a 31.6 $\times$ lower cost using NVMe SSDs and at a 300.5 $\times$ lower cost using HDDs than the system using 0.93 petabytes of DDR4 DRAM. Even though the system with large main memory is faster, the slowdown of SnusQS is tolerable as long as the cost-effectiveness is huge.

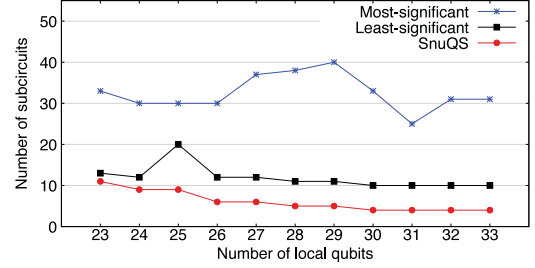
5.7 Memory Overlays versus Demand Paging

We compare the memory overlays of SnusQS and the demand paging by the operating system used in Qiskit and QuEST. We simulate two gates H and CZ . H targets q_0 and CZ targets q_0 and q_{40} in a 41-qubit system. We use the CPU system in this experiment, and the total size of the virtual memory space is set to 32TB to support 41 qubits.

SnusQS achieves 16.59 $\times$ and 16.06 $\times$ speedups for H , and 14.58 $\times$ and 13.25 $\times$ speedups for CZ , over QuEST and Qiskit, respectively. In the worst case, QuEST takes 23.89 hours to simulate H . We estimate the simulation time of QSC-41-26 using the simulation time of CZ by Qiskit (8.86 hours) multiplied by the number of gates

Table 7. Simulation Time of a 6x5 depth 1+24+1 QSC and a 30-qubit QFT circuit (in seconds)

Simulator	QSC	QFT
QuEST	12.08	12.34
Qiskit (Aer)	25.99	21.42
Cirq (qsim)	6.12	11.56
Cirq (cuQuantum)	6.66	10.44
SnusQS	5.99	9.91

**Figure 13.** The number of sub-circuits of a 7x6 QSC when the number of local qubits vary.

(613) without quantum circuit optimizations. It takes about 226 days for Qiskit to simulate QSC-41-26. Assuming the exponential scalability, we see that QSC-42-26 would take 452 days and QSC-42-50 would take more than two years. It indicates that the full-state simulation of large quantum circuits using the demand paging in the operating system is infeasible.

5.8 Circuit Optimizations

To show that SnusQS is fast even for small quantum circuits, we compare the simulation time of 30-qubit quantum circuits between QuEST[28], Qiskit Aer[9], Cirq qsim and Cirq using cuQuantum[1], and SnusQS GPU simulator. We use two benchmarks: a depth 1+24+1 6x5 QS circuit and a 30-qubit QFT circuit. The circuits require 16GB of memory space in double precision floating point format which is less than the GPU memory size. Thus, SnusQS does not use overlays and has no I/O overhead. Table 7 shows that SnusQS is the fastest for both benchmarks.

5.9 Permutation Selection

The smaller the number of sub-circuits, the smaller the amount of storage I/O. Figure 13 shows the number of sub-circuits as the

Table 8. Comparison between the state-of-the-art on Cori II[25] and SnusQS using GPUs and HDDs

	The state-of-the-art on Cori II	SnusQS using GPUs and HDDs
# of Nodes	4,096	1
Compute devices per node	1 $\times$ Intel Xeon Phi 7250 (KNL)	1 $\times$ AMD EPYC 7702P 4 $\times$ NVIDIA RTX 3090
Memory type and capacity per node	Main: 96 GB DDR4 KNL: 16 GB MCDRAM (on-package)	Main: 512GB DDR4 GPU: 4 $\times$ 24GB GDDR6X
Rpeak (double precision)	12,470 TFLOPS	2.22 TFLOPS
Rmax (double precision)	298,567 GFLOPS	672.36 GFLOPS
Simulation time	79.53 seconds	35,316.01 seconds
FP efficiency	2.39 %	30.29 %
Single-node execution time	25.52 hours (conservatively estimated)	9.81 hours

number of local qubits (*i.e.*, the main memory size) varies for QSC-42-50. We compare SnuQS permutation selection described at the end of Section 3.4 with two simple methods: *Most-significant* and *Least-significant*. *Most-significant* (*Least-significant*) always swaps non-local qubit positions with the most (least) significant local qubit positions. These two methods are similar to the gate swapping method proposed by Häner *et al.*[25] to optimize communication. The result shows that our method always outperforms *Most-significant* and *Least-significant*. The trend for QSC-42-26 is similar to that of QSC-42-50. Since the QFT circuits always produce two sub-circuits, we omit the result for them.

5.10 Comparison with the SOTA Full-state Simulator

We compare SnuQS with the state-of-the-art full-state simulator on Cori II[25], say CORI. It is hard to compare them directly because they have different system configurations. Instead, we compare their floating-point (FP) efficiency and single-node execution time.

FP efficiency. We first compare the *FP efficiency*, *i.e.*, the achieved FLOPS (R_{max}) divided by the theoretical peak FLOPS (R_{peak}), in the system. R_{max} is computed by

$$R_{max} = \frac{\# \text{ of FP operations for simulation}}{\text{The simulation time in seconds}}. \quad (9)$$

For the comparison, we use a depth $1+24+1$, 7×6 -qubit (QSC-42-26) QSC. We compute the total number of FP operations for the simulation as follows. QSC-42-26 requires 2^{42} complex amplitudes and consists of 84 H gates, 94 $\sqrt{X}$ gates, 108 $\sqrt{Y}$ gates, 126 T gates, and 213 CZ gates. As described in Section 2, a non-diagonal one-qubit gate (*e.g.*, H , $\sqrt{X}$, and $\sqrt{Y}$) takes two complex multiplications and one complex addition to update an amplitude. A diagonal one-qubit gate (*e.g.*, T) takes a single complex multiplication to update an amplitude. Similarly, a controlled diagonal two-qubit gate (*e.g.*, CZ) takes a single complex multiplication to update an amplitude, but it only updates half of the amplitudes. A complex multiplication requires six FP operations, and a complex addition requires two FP operations. Thus, a simple calculation shows that the simulation requires 23,745.05 FP operations. Meanwhile, CORI and SnuQS take 79.53 seconds[25] and 9.81 hours, respectively. The R_{max} of CORI and SnuQS is 298,567 GFLOPS and 672.36 GFLOPS, respectively.

Even though Häner and Steiger[25] use a depth $1+24+1$, 7×6 -qubit QSC for their simulation on Cori II, we note that the exact gate types used in CORI may be different from the QSC-42-26 used by SnuQS because the information about their depth $1+24+1$, 7×6 -qubit QSC is not available. Since the types of single-qubit gates ($\sqrt{X}$, $\sqrt{Y}$, and T gates) are randomly selected by the QSC generation rule[13], the total number of FP operations computed above for QSC-42-26 may not be exact for CORI. However, the expected number of each gate type in different instances of depth $1+24+1$, 7×6 -qubit QSCs is guaranteed to be the same by the generation rule[13].

Table 8 shows the comparison result. CORI uses 4,096 nodes, each of which has an Intel Xeon Phi 7250 (KNL). The R_{peak} of Cori II is 12,470 TFLOPS. SnuQS uses a single node containing four NVIDIA GeForce RTX 3090 GPUs. The R_{peak} of each GPU is 556 GFLOP/s, and thus the R_{peak} of the SnuQS system is 2.22 TFLOPS.

SnuQS achieves much better FP efficiency than CORI. CORI achieves 2.39% of FP efficiency while SnuQS achieves 30.29%. Note that the FP efficiency is relatively low in both cases because the Schrödinger-style simulation algorithm is memory-intensive, and communication (or I/O) becomes the performance bottleneck.

Execution time. Now we compare the single-node execution time. We define the *single-node execution time* to be the minimal simulation time when we assume a single-node configuration for both CORI and SnuQS. The simulation by CORI with 4,096 nodes takes 79.53 seconds, and it spends 71.8% of the execution time on communication between nodes[25]. Then, the execution time without any communication is $79.53 \times 28.2\% = 22.43$ seconds for each node. Thus, a conservative estimation of the execution time of the state-of-the-art simulator on a single Cori II node is $22.43 \times 4,096 = 91,873.28$ seconds, and it is equivalent to 25.52 hours. The single-node configuration is equivalent to the configuration in which a single Cori II node has the main memory capacity equivalent to that of the entire main memory in 4,096 nodes. Note that the single-node configuration is practically infeasible, and we use it just to compare SnuQS and CORI conservatively.

On the contrary, SnuQS takes 9.81 hours with GPUs and HDDs. SnuQS achieves better single-node performance than the state-of-the-art. Since the estimation of the single-node performance of the state-of-the-art is conservative (*i.e.*, excluding communication time), we expect that SnuQS outperforms the state-of-the-art full-state simulator significantly.

SnuQS outperforms the state-of-the-art for the following two reasons. One is that since gate operations in the full-state simulation are memory-bound[25], the memory bandwidth of the target systems is the most critical performance factor. The GPU memory bandwidth (GDDR6X) is much higher than the main memory (DDR4) or on-package memory (MCDDR4) bandwidth in the Cori II node. We measure the effective memory bandwidth using Empirical Roofline Toolkit[3] in the SnuQS-GPU system. Each GPU memory bandwidth is 819.2 GB/s, and $4 \times 819.2 = 3,276.8$ GB/s for all the GPUs. The DDR4 main memory bandwidth is 142.8 GB/s. The empirical on-package MCDRAM memory bandwidth of KNL is not available, but its theoretical maximum bandwidth is 496 GB/s, which is much smaller than the total GPU bandwidth. Note that Häner and Steiger[25] use the on-package MCDRAM as a cache of the main memory. The other reason is that SnuQS effectively hides CPU-GPU communication, and hence there is no data transfer overhead of offloading gate operations to GPUs. SnuQS overlaps gate operations (CUDA kernel computations) and CPU-GPU communication through triple buffering for each GPU. Moreover, the storage I/O operations are also effectively hidden and overlap with the gate operations in the GPUs.

6 Related Work

In this section, we summarize the previous work and compare SnuQS to the state-of-the-art cluster-based simulators. We also show how different the I/O optimization techniques in SnuQS are from other techniques.

6.1 Simulation Techniques

Full-state simulation and amplitude sampling are two dominant methods explored in many studies as described in Section 1. There are other simulation methods than full-state simulation and amplitude sampling, such as stabilizer circuit simulation[8, 24, 26, 34] and approximation techniques[13, 43]. The stabilizer circuits cannot represent most quantum algorithms since it does not use a universal quantum gate set. The approximation techniques are helpful to reduce memory and computation requirements, but we still need exact simulation until a stable quantum computer is available.

Optimizations in full-state simulation. To improve simulation performance, some full-state simulators apply vectorization[1, 9, 16], support GPU acceleration[1, 9, 28], and perform distributed processing[28]. Also, they apply many optimization techniques, such as cache blocking[41] gate scheduling, and gate fusion[1, 9]. Fatima *et al.*[22] propose a highly optimized simulation algorithm that uses the single-precision floating-point format and reduces expensive floating-point multiplications. Häner *et al.* [25] emphasize the importance of gate scheduling in full-state simulation. They distinguish local and global qubits and propose swapping qubits to reduce the communication overhead between different nodes in a cluster. Pednault *et al.*[37] adopt a similar idea in the tensor network framework. They propose a more systematic gate scheduling algorithm. Unlike these approaches, the goal of SnuQS is to reduce the amount of storage I/O to increase performance and scalability.

Handling the exponential growth. A typical approach to overcome the exponential growth of the main memory space in full-state simulation is using large-scale clusters (*e.g.*, supercomputers). Jones *et al.*[28] use 2,048 CRAY XC30 ARCHER nodes to simulate a 38-qubit quantum circuit, Smelyanskiy *et al.*[41] use 1,000 Stampede nodes to simulate a 40-qubit circuit, and Häner *et al.*[25] uses 8,192 Cori II nodes to simulate a 45-qubit (0.5 petabyte-scale) circuit. They mainly focus on operation and communication optimization between nodes[14, 25, 32, 37, 38, 41].

SnuQS solves the exponential growth problem by exploiting storage devices. To the best of our knowledge, SnuQS is the first work using storage devices to scale the full-state simulation of quantum circuits except for analytical estimation using the hybrid method[37].

Cluster-based simulator versus SnuQS. Unlike previous cluster-based approaches, SnuQS is a storage-based simulator. Both cluster-based simulators, and it tries to enlarge the simulation limit, but they differ in many ways. The primary goal of SnuQS is to achieve cost-effective quantum circuit simulation using storage devices, while the primary purpose of the cluster-based simulators is to reduce the simulation time. Moreover, the cluster-based simulators exploit distributed data layout and communication optimization techniques, while SnuQS proposes storage layout for the full-state simulation and I/O optimization techniques. The evaluation result of SnuQS shows that it is scalable and cost-effective to use storage devices for quantum circuit simulation.

6.2 Circuit Partitioning Techniques

Both the state-of-the-art simulator[25] and SnuQS are based on circuit partitioning techniques. Circuit partitioning is a well-known technique as a variant of graph partitioning. However, the partitioning algorithm's details heavily depend on the target system and the simulation method. For example, Pednault *et al.*[38] allow horizontal circuit partitioning because they use the tensor network contraction algorithm. Häner *et al.* [25] manually clusters a sequence of quantum gates to reduce all-to-all communication. SnuQS uses an automatic circuit partitioning technique based on its qubit permutation and amplitude layout. Moreover, SnuQS prefetches amplitudes to overlap computation and I/O. SnuQS combines the circuit partitioning technique with the three-step qubit permutation to guarantee that the I/O operations are grouped in a contiguous block, and the block size is large enough to achieve high I/O bandwidth.

6.3 I/O Optimization Techniques

There exist many optimization techniques to improve I/O performance, such as data prefetching, parallel I/O, and data layout transformation [30, 44]. RAID [36] is also an example that accesses multiple storage devices in parallel to increase I/O bandwidth (*e.g.*, RAID 0) and secure data integrity. Even though I/O optimization techniques for classical scientific applications are widespread, it has not been explored to apply them to quantum circuit simulation.

SnuQS leverages existing I/O optimization techniques to boost quantum circuit simulation using storage devices. However, SnuQS applies a combination of customized I/O optimization techniques specifically to quantum circuit simulation:

- It uses a unique data layout for state amplitudes in the main memory and storage devices for memory overlays to exploit I/O-level parallelism. Based on the amplitude layout, it applies data prefetching and accesses storage spaces in parallel.
- It uses an automatic sub-circuit partitioning algorithm that reduces the I/O frequency and size. Using the sub-circuit partitioning, SnuQS reads and writes the entire storage space only once at each sub-circuit boundary.
- It uses a three-step permutation technique to transform data layout efficiently and systematically. The three-step permutation technique allows any arbitrary data layout to be transformed into an appropriate amplitude layout. It prevents I/O block size from being too small to improve I/O bandwidth.

7 Conclusions

The overlay-based full-state quantum circuit simulation framework, SnuQS, exploits storage devices, such as HDDs and NVMe SSDs, to enlarge the available main memory capacity at a small cost. The experimental result indicates that the in-memory simulation performance of SnuQS is comparable to the state-of-the-art simulators, such as QuEST, Qiskit Aer, and Cirq qsim. Using a single node machine with four NVIDIA GTX 3090 GPUs and 80 2TB HDDs, simulations of a 42-qubit QSC of depth $1+48+1$ and a QFT of depth 84 take 16.72 and 7.44 hours, respectively. Using ten 4TB NVMe SSDs, simulations of a 39-qubit QSC of depth $1+48+1$ and a QFT of depth 78 take 41.4 and 9.6 minutes, respectively.

Assuming the maximum capacity (16TB) of commercially available HDDs, we can attach a total of 1,280TB to the current I/O subsystem with no performance penalty. Using this configuration, we estimate that SnuQS and the I/O subsystem together can simulate a 45-qubit QSC of depth $1+48+1$ and a 45-qubit QFT of depth 84 in 133.76 hours (5.57 days) and 59.52 hours (2.48 days), respectively at about $300\times$ lower cost than the DDR4 DRAM main-memory-only system.

Acknowledgement

This work was supported in part by the National Research Foundation of Korea (NRF) grants (No. NRF-2019M3E4A1080386, End-to-End Quantum Software Stack for Virtual Execution), by the BK21 Plus program for BK21 FOUR Intelligence Computing (Dept. of Computer Science and Engineering, SNU, No. 4199990214639) through NRF, and by the Institute for Information & communications Technology Promotion (IITP) grant (No. 2018-0-00581, CUDA Programming Environment for FPGA Clusters), all funded by the Ministry of Science and ICT (MSIT) of Korea. ICT at Seoul National University provided research facilities for this study.

References

- [1] 2022. *Cirq*. <https://cirq.readthedocs.io/en/stable/>
- [2] 2022. *Efficient IO with io_uring*. https://kernel.dk/io_uring.pdf
- [3] 2022. *Empirical Roofline Tool (ERT)*. <https://crd.lbl.gov/departments/computer-science/par/research/roofline/software/ert/>
- [4] 2022. *Fio*. <https://github.com/axboe/fio>
- [5] 2022. TOP500 November 2021. <https://www.top500.org/lists/top500/2021/11/>
- [6] 2022. v-Color 128GB(64GBx2) 3200MHz DDR4 ECC R-DIMM 4Gx4 2Rx4 ECC Registered DIMM Using hynix IC Dual Rank for Server/Workstation/Data Center Memory(TR464G32D422K). https://www.amazon.com/v-Color-3200MHz-Registered-Workstation-TR464G32D422K/dp/B08PXRQX1/ref=sr_1_3?dchild=1&keywords=v+color+128GB&qid=1628789850&sr=8-3
- [7] 2022. Western Digital WD Ultrastar DC HC550 16TB SATA 6Gb/s 7200RPM 3.5-Inch Data Center HDD (WUH721816ALE6L4). https://www.amazon.com/Western-Digital-Ultrastar-3-5-Inch-WUH721816ALE6L4/dp/B08DHHMZBF/ref=sr_1_8?dchild=1&keywords=HDD+16TB&qid=1628787716&sr=8-8
- [8] Scott Aaronson and Daniel Gottesman. 2004. Improved simulation of stabilizer circuits. *Physical Review A* 70, 5 (2004), 052328.
- [9] Héctor Abraham, AduOfiei, Rochisha Agarwal, Ismail Yunus Akhalwaya, Gadi Aleksandrowicz, Thomas Alexander, Eli Arbel, Abraham Asfaw, Carlos Azaustre, AzizNgoueya, et al. 2019. Qiskit: An Open-source Framework for Quantum Computing. <https://doi.org/10.5281/zenodo.2562110>
- [10] Adriano Barenco, Charles H Bennett, Richard Cleve, David P DiVincenzo, Norman Margolus, Peter Shor, Tycho Sleator, John A Smolin, and Harald Weinfurter. 1995. Elementary gates for quantum computation. *Physical Review A* 52, 5 (1995), 3457.
- [11] Rami Barends, Julian Kelly, Anthony Megrant, Andrzej Veitia, Daniel Sank, Evan Jeffrey, Ted C White, Josh Mutus, Austin G Fowler, Brooks Campbell, et al. 2014. Superconducting quantum circuits at the surface code threshold for fault tolerance. *Nature* 508, 7497 (2014), 500–503.
- [12] Charles H Bennett, David P DiVincenzo, John A Smolin, and William K Wootters. 1996. Mixed-state entanglement and quantum error correction. *Physical Review A* 54, 5 (1996), 3824.
- [13] Sergio Boixo, Sergei V Isakov, Vadim N Smelyanskiy, Ryan Babbush, Nan Ding, Zhang Jiang, Michael J Bremner, John M Martinis, and Hartmut Neven. 2018. Characterizing quantum supremacy in near-term devices. *Nature Physics* 14, 6 (2018), 595–600.
- [14] Sergio Boixo, Sergei V Isakov, Vadim N Smelyanskiy, and Hartmut Neven. 2017. Simulation of low-depth quantum circuits as complex undirected graphical models. *arXiv preprint arXiv:1712.05384* (2017).
- [15] Gilles Brassard. 1996. Teleportation as a quantum computation. *arXiv preprint quant-ph/9605035* (1996).
- [16] Michael Broughton, Guillaume Verdon, Trevor McCourt, Antonio J Martinez, Jae Hyeon Yoo, Sergei V Isakov, Philip Massey, Murphy Yuezhen Niu, Ramin Halavati, Evan Peters, et al. 2020. Tensorflow quantum: A software framework for quantum machine learning. *arXiv preprint arXiv:2003.02989* (2020).
- [17] Steven WD Chien, Stefano Markidis, Chaitanya Prasad Sishtla, Luis Santos, Pawel Herman, Sai Narasimhamurthy, and Erwin Laure. 2018. Characterizing deep-learning I/O workloads in TensorFlow. In *2018 IEEE/ACM 3rd International Workshop on Parallel Data Storage & Data Intensive Scalable Computing Systems (PDSW-DISCS)*. IEEE, 54–63.
- [18] Don Coppersmith. 2002. An approximate Fourier transform useful in quantum factoring. *arXiv preprint quant-ph/0201067* (2002).
- [19] Leonardo Dagum and Ramesh Menon. 1998. OpenMP: an industry standard API for shared-memory programming. *Computational Science & Engineering, IEEE* 5, 1 (1998), 46–55.
- [20] Hans De Raedt, Fengping Jin, Dennis Willsch, Madita Willsch, Naoki Yoshioka, Nobuyasu Ito, Shengjun Yuan, and Kristel Michielsen. 2019. Massively parallel quantum computer simulator, eleven years later. *Computer Physics Communications* 237 (2019), 47–61.
- [21] Koen De Raedt, Kristel Michielsen, Hans De Raedt, Binh Trieu, Guido Arnold, Marcus Richter, Th Lippert, Hiroshi Watanabe, and Nobuyasu Ito. 2007. Massively parallel quantum computer simulator. *Computer Physics Communications* 176, 2 (2007), 121–136.
- [22] Aneeqa Fatima and Igor L Markov. 2021. Faster Schrödinger-style simulation of quantum circuits. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 194–207.
- [23] Haohuan Fu, Yuling Yang, Jiawei Song, Pengpeng Zhao, Zhen Wang, Dajia Peng, Huarong Chen, Chu Guo, Heliang Huang, Wenzhao Wu, et al. 2021. Closing the “quantum supremacy” gap: achieving real-time simulation of a random quantum circuit using a new Sunway supercomputer. *arXiv preprint arXiv:2110.14502* (2021).
- [24] Daniel Gottesman. 1998. The Heisenberg representation of quantum computers. *arXiv preprint quant-ph/9807006* (1998).
- [25] Thomas Häner and Damian S Steiger. 2017. 0.5 petabyte simulation of a 45-qubit quantum circuit. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–10.
- [26] Stefan Hillmich, Igor L Markov, and Robert Wille. 2020. Just like the real thing: Fast weak simulation of quantum computation. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [27] Yipeng Huang and Margaret Martonosi. 2019. Statistical assertions for validating patterns and finding bugs in quantum programs. In *Proceedings of the 46th International Symposium on Computer Architecture*. 541–553.
- [28] Tyson Jones, Anna Brown, Ian Bush, and Simon C Benjamin. 2019. QuEST and high performance simulation of quantum computers. *Scientific reports* 9, 1 (2019), 1–11.
- [29] Richard Jozsa. 1998. Quantum algorithms and the Fourier transform. *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences* 454, 1969 (1998), 323–337.
- [30] Sidharth Kumar, Avishek Saha, Venkatram Vishwanath, Philip Carns, John A Schmidt, Giorgio Scorzelli, Hemanth Kolla, Ray Grout, Robert Latham, Robert Ross, et al. 2013. Characterization and modeling of pidx parallel I/O for performance optimization. In *Proceedings of the international conference on high performance computing, networking, storage and analysis*. 1–12.
- [31] Jianwei Li, Wei-keng Liao, Alok Choudhary, Robert Ross, Rajeev Thakur, William Gropp, Robert Latham, Andrew Siegel, Brad Gallagher, and Michael Zingale. 2003. Parallel netCDF: A high-performance scientific I/O interface. In *SC'03: Proceedings of the 2003 ACM/IEEE conference on Supercomputing*. IEEE, 39–39.
- [32] Riling Li, Bujiao Wu, Mingsheng Ying, Xiaoming Sun, and Guangwen Yang. 2018. Quantum supremacy circuit simulation on Sunway TaihuLight. *arXiv preprint arXiv:1804.04797* (2018).
- [33] Igor L Markov, Aneeqa Fatima, Sergei V Isakov, and Sergio Boixo. 2018. Quantum supremacy is both closer and farther than it appears. *arXiv preprint arXiv:1807.10749* (2018).
- [34] Igor L Markov, Aneeqa Fatima, Sergei V Isakov, and Sergio Boixo. 2020. Massively parallel approximate simulation of hard quantum circuits. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [35] NVIDIA, Péter Vingelmann, and Frank H.P. Fitzek. 2020. CUDA, release: 10.2.89. <https://developer.nvidia.com/cuda-toolkit>.
- [36] David A Patterson, Garth Gibson, and Randy H Katz. 1988. A case for redundant arrays of inexpensive disks (RAID). In *Proceedings of the 1988 ACM SIGMOD international conference on Management of data*. 109–116.
- [37] Edwin Pednault, John A Gunnels, Giacomo Nannicini, Lior Horeish, Thomas Magerlein, Edgar Solomonik, Erik W Draeger, Eric T Holland, and Robert Wisnieff. 2017. Pareto-Efficient Quantum Circuit Simulation Using Tensor Contraction Deferral. *arXiv preprint arXiv:1710.05867* (2017).
- [38] Edwin Pednault, John A Gunnels, Giacomo Nannicini, Lior Horeish, and Robert Wisnieff. 2019. Leveraging secondary storage to simulate deep 54-qubit sycamore circuits. *arXiv preprint arXiv:1910.09534* (2019).
- [39] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J Love, Alán Aspuru-Guzik, and Jeremy L O’Brien. 2014. A variational eigenvalue solver on a photonic quantum processor. *Nature communications* 5, 1 (2014), 1–7.
- [40] John Preskill. 2018. Quantum Computing in the NISQ era and beyond. *Quantum* 2 (2018), 79.
- [41] Mikhail Smelyanskiy, Nicolas PD Sawaya, and Alán Aspuru-Guzik. 2016. qHiPSTER: The quantum high performance software testing environment. *arXiv preprint arXiv:1601.07195* (2016).
- [42] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (2009), 65–76.
- [43] Xin-Chuan Wu, Sheng Di, Emma Maitreyee Dasgupta, Franck Cappello, Hal Finkel, Yuri Alexeev, and Frederic T Chong. 2019. Full-state quantum circuit simulation by using data compression. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–24.
- [44] Yanlong Yin, Surendra Byna, Huaiming Song, Xian-He Sun, and Rajeev Thakur. 2012. Boosting application-specific parallel I/O optimization using IOSIG. In *2012 12th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (ccgrid 2012)*. IEEE, 196–203.
- [45] Sangjin Yoo and Dongkun Shin. 2020. Reinforcement Learning-Based SLC Cache Technique for Enhancing SSD Write Performance. In *12th {USENIX} Workshop on Hot Topics in Storage and File Systems (HotStorage 20)*.